

1. Revise Spring bean life cycle
2. Different modes of wiring

3. Continue to autowiring

4.

Hybrid approach

(Lesser XML n majority annotations)

Understand this !

Field level D.I (no setters n no parameterized ctor)

1. `@Autowired` `//=> autowire=byType` `bydefault required=true`

private Teacher myTeacher;

In case of no match - SC throws the exception - `NoSuchBeanDefinitionException`

In case multiple matches - `NoUniqueBeanDefinitionExc`

In case exact match - no excs --B.L works using dependencies .

2. `@Autowired(required=false)` `//=> autowire=byType`

private Teacher myTeacher;

In case of no match - B.L failure (`NPE`)

In case multiple matches - `NoUniqueBeanDefinitionExc`

In case exact match - no excs --B.L works using dependencies .

3. `@Autowired` `//=> autowire=byType`

`@Qualifier("eng")` `//=> byName`

private Teacher myTeacher;

In case of no match - SC throws the exception - `NoSuchBeanDefinitionException`

In case multiple matches - `NoUniqueBeanDefinitionExc`

In case exact match - no excs --B.L works using dependencies .

5. Enter MVC is DESIGN PATTERN

What is MVC ?

Model-View-Controller --Standard design pattern , meant for clear cut separation of concerns(=tasks=responsibilities)

In Custom implementation(done by programmer) of the MVC pattern :

Model -- Java Bean (conversational state holder + B.L supplier) & POJOs

View layer --JSP , Thymeleaf/Velocity/Angular/React/Vue/Next JS

Represents UI / presentation logic (processing requests & generating response)

Controller -- Typically a servlet(used in Spring MVC) or a filter(used in Struts 2 framework)

Manages navigation(routing) & beans.

Front Controller --

It's an additional design pattern , along with MVC -

To ensure that ANY request coming from ANY client , for this web app , should be intercepted by a common gate keeper(or a centralized dispatcher)

It will dispatch client request to further components , based upon nature of the req.

MVC flow (without spring)

refer : MVC diagrams

Instead of creating a custom implementation of MVC n Front Controller , Spring framework , supplies it in a ready made manner, to simplify java development.

Enter Spring MVC

(Refer : "day11-data\day11_help\Spring MVC\steps for spring mvc.txt")

Objectives

1. Test Spring MVC Flow
 2. Deliver/render the index.jsp from SC (using spring MVC architecture)
 3. Test Model n View
 4. Test Model Map
 5. Enter Case study (Blogs Management System)
- 5.1 User Sign in

Handling request params in Spring MVC

Spring MVC integration with Hibernate

A diagram for steps n code together for complete understanding.